



ADDIS ABABA UNIVERSITY

DEPARTMENT OF PHYSICS

Computational Physics

Assignment Problems (10%)

Regular Program, 2025/2026 A.Y.

Submitted by:

Getabalew Manegerew

ID: UGR/6981/15

Instructor:

Dr. Kenate

January 17, 2026

Problem 1: Chapter One Exercise #7

Problem Statement

7. Write a program which solves a quadratic equation: $ax^2 + bx + c = 0$. The program code need to return real values if the solution is real and complex values if the solution is complex. Logically incorporate all the procedures, (i.e, make module, subroutine, and function), into your program code.

Solution

```
1 PROGRAM problem1
2   IMPLICIT NONE
3   REAL :: a=1, b=2, c=3
4   COMPLEX :: x1, x2
5
6
7   CALL quadratic(a,b,c,x1,x2)
8   PRINT*, "----- Solutions in complex number format
9   -----"
10  PRINT*, "The two roots are", x1
11  PRINT*, "And ", x2
12
13 END PROGRAM problem1
14
15 SUBROUTINE quadratic(a,b,c,x1,x2)
16   IMPLICIT NONE
17   REAL, INTENT(IN) :: a, b, c
18   COMPLEX, INTENT(INOUT) :: x1, x2
19
20   IF ((b**2-4*a*c)>=0) THEN
21     x1 = (-b - sqrt(b**2-4*a*c))/(2*a)
22     x2 = (-b + sqrt(b**2-4*a*c))/(2*a)
23   ELSE
24     x1 = cmplx(-b/(2*a), - sqrt(4*a*c-b**2)/(2*a))
25     x2 = cmplx(-b/(2*a), sqrt(4*a*c-b**2)/(2*a))
26   END IF
27 END SUBROUTINE quadratic
```

Listing 1: Fortran code for Problem 1

Problem 2: Chapter One Exercise #19

Problem Statement

19. Assume a ball bounces back and forth elastically and in a vertical direction. Every time the ball completes one cycle, the maximum vertical height it can reach reduces by $1/2$. Assuming the ball initially can reach a maximum height of 50 m, determine the total vertical distance that it travels before eventually resting on the reference ground. Show your result in graph plots.

Solution

```
1 PROGRAM problem2
2   IMPLICIT NONE
3   INTEGER, PARAMETER :: N=500
4   REAL :: h0=50, h(0:N), r=0.5
5   INTEGER :: k
6   h(0)=h0
7
8   DO k=1, N
9       h(k) = r*h(k-1)
10      PRINT*, h(k)
11  END DO
12
13
14
15  OPEN(UNIT=10, FILE="ass_problem2.csv", POSITION="APPEND",
16       ACCESS="SEQUENTIAL", STATUS="REPLACE")
17  DO k=0, N
18      WRITE(UNIT=10, FMT="(I6, f11.8)") k, h(k)
19  END DO
20  CLOSE(UNIT=10)
21
22
23 END PROGRAM problem2
```

Listing 2: Fortran code for Problem 2

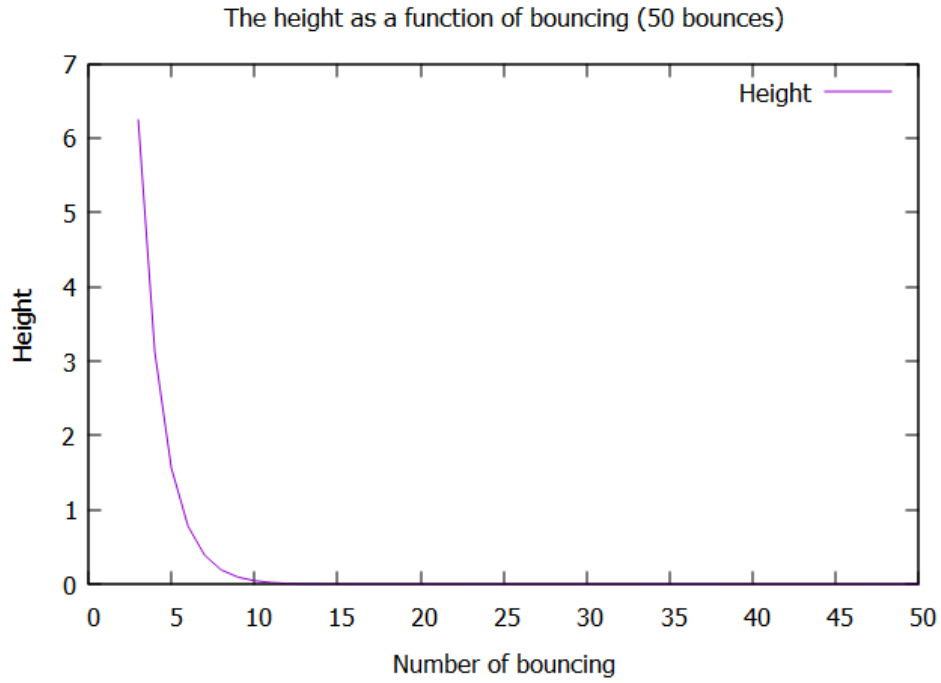


Figure 1: Simulation of the bouncing ball motion. The graph illustrates the decay of the maximum vertical height, which reduces by a factor of $1/2$ after each cycle ($h_n = 0.5h_{n-1}$), starting from an initial height of 50 m. (I used 50 number of bouncings in this graph)

Problem 3: Track Vehicle Integration

Problem Statement

A track vehicle moves at a speed of 10 m/s at $t = 0$. It accelerates at $a = 3 \text{ m/s}^2$ until $t = 3 \text{ s}$. Determine the total distance travelled using **Trapezoid** and **Simpson** methods for $n = 1, 3, 5, 10$ slices.

$$S = \int_0^3 (v_0 + at) dt$$

Fortran Code

```
1  ! =====
2  MODULE integrator
3      IMPLICIT NONE
4
5
6      CONTAINS
7      SUBROUTINE simpson(t, simp)
8          REAL, EXTERNAL :: f
9          REAL :: v0=10, a=3, t(0:), simp, t0=0, tf=3, dt
10         INTEGER :: i, k, N
11         N = size(t) - 1
12         dt = (tf - t0)/N
13         t(0) = t0
14
15         DO k=1, N
16             t(k) = t(k-1) + dt
17         END DO
18
19         IF (MOD(N,2)==0) THEN
20
21             DO i=1, N-1, 2
22                 simp = simp + dt*(4*f(t(i)) + f(t(i-1)) + f(t(i+1)))/3
23             END DO
24         ELSE
25
26             DO i=1, N-2, 2
27                 simp = simp + dt*(4*f(t(i)) + f(t(i-1)) + f(t(i+1)))/3
28             END DO
29             simp = simp + dt*(-f(t(N-2)) + 8*f(t(N-1)) + 5*f(t(N)))/12
30         END IF
31     END SUBROUTINE simpson
32
33     SUBROUTINE trapziod(t, trap)
34         REAL, EXTERNAL :: f
35         REAL :: v0=10, a=3, t(0:), trap, t0=0, tf=3, dt
36         INTEGER :: i, k, N
37         N = size(t) - 1
38         trap = 0
```

```

39      dt = (tf - t0)/N
40      t(0) = t0
41      DO k=1, N
42          t(k) = t(k-1) + dt
43      END DO
44
45
46      DO i=1, N
47          trap = trap + dt*(f(t(i)) + f(t(i-1)))/2
48      END DO
49  END SUBROUTINE trapziod
50
51  END MODULE integrator
52
53
54  PROGRAM problem3
55      USE integrator
56      IMPLICIT NONE
57      REAL, EXTERNAL :: f
58      INTEGER, PARAMETER :: N1=1, N3=3, N5=5, N10=10
59      REAL :: v0=10, a=3, dt, t0=0, tf=3
60      REAL:: x1(0:N1), x3(0:N3), x5(0:N5), x10(0:N10), trap1, trap3
61          , trap5, trap10, simp1, simp3, simp5, simp10
62
63      CALL trapziod(x1, trap1)
64      CALL trapziod(x3, trap3)
65      CALL trapziod(x5, trap5)
66      CALL trapziod(x10, trap10)
67      CALL simpson(x1, simp1)
68      CALL simpson(x3, simp3)
69      CALL simpson(x5, simp5)
70      CALL simpson(x10, simp10)
71
72      WRITE(*,FMT="(A16, 4X, A20, 4X, A20)") "Number of Slices",
73          "Result from Trapziod", "Result from Simpson"
74      WRITE(UNIT=6, FMT="(I3, 20X, F6.3, 20X, F6.3)") N1, trap1,
75          simp1
76      WRITE(UNIT=6, FMT="(I3, 20X, F6.3, 20X, F6.3)") N3, trap3,
77          simp3
78      WRITE(UNIT=6, FMT="(I3, 20X, F6.3, 20X, F6.3)") N5, trap5,
79          simp5
80      WRITE(UNIT=6, FMT="(I3, 20X, F6.3, 20X, F6.3)") N10,
81          trap10, simp10
82
83      PRINT*, simp5
84
85  END PROGRAM problem3

```

```

84
85 FUNCTION f(t) RESULT(y)
86     IMPLICIT NONE
87     REAL :: t, y, v0=10, a=3
88     y = v0 + a*t
89 END FUNCTION f=====

```

Listing 3: Fortran code for Numerical Integration

Results

The calculated total distance for different numbers of slices (n) is tabulated below:

Table 1: Numerical Integration Results for Total Distance Travelled ($t = 3\text{s}$)

Number of Slices (n)	Result from Trapezoid (m)	Result from Simpson (m)
1	43.500	41.250
3	43.500	43.500
5	43.500	43.500
10	43.500	43.500

Exact Analytical Value: 43.500 m

Problem 4: Vertical Particle Motion

Problem Statement

A particle is thrown vertically up from a building top with $v_0 = 10 \text{ m/s}$.

- Find the time t when the particle strikes back the building.
- Find the displacement S travelled during this interval.

Given: $g = 10 \text{ m/s}^2$. Use a root finder for t and an integration solver for S .

Fortran Code

```
1  ! =====
2  MODULE DRIVER
3      IMPLICIT NONE
4      CONTAINS
5      SUBROUTINE bisection(t0, tb, tol)
6          REAL, EXTERNAL :: f
7          REAL :: t0, tb, t, tol
8          INTEGER :: m
9          t = (t0+tb)/2
10
11      DO WHILE(abs(f(t))>tol)
12          t = (t0+tb)/2
13          IF ((f(t0)*f(t))<0) THEN
14              tb=t
15          ELSE
16              t0=t
17          END IF
18          m=m+1
19      END DO
20      END SUBROUTINE bisection
21
22
23      SUBROUTINE simpson(t, simp, N)
24          REAL, EXTERNAL :: v
25          REAL :: v0=10, t(0:), simp, t0=0, tf=1, dt
26          INTEGER :: i, k, N
27
28          dt = (tf - t0)/N
29          t(0) = t0
30          DO k=1, N
31              t(k) = t(k-1) + dt
32          END DO
33
34          DO i=1, N-1, 2
35              simp = simp + dt*(4*v(t(i)) + v(t(i-1)) + v(t(i+1)))/3
36          END DO
37      END SUBROUTINE simpson
38
```



```

39
40 END MODULE DRIVER
41
42
43 PROGRAM problem3_2
44     USE DRIVER
45     IMPLICIT NONE
46     REAL, EXTERNAL :: f
47     INTEGER, PARAMETER :: N=10000
48     REAL :: t0=1, tb=5, tol=2e-5, dt=1e-5, simp, t(0:N)
49
50
51     CALL bisection(t0,tb, tol)
52     CALL simpson(t, simp, N)
53
54     ! When the particle hit the ground again,
55     ! it reached again to the origin of our frame of reference
56     ! thus we can use bisection method at what time this happend
57     ! Appart that in physics term the displacement of the particle
58     ! is 0 when it hits the ground
59     ! however, we can determine the total distance by using simpson
60     ! rule for one half of the flight and multiply it by 2
61
62     WRITE(UNIT=6, FMT="(A40, X, F4.2, X, A8)") "The particle hit the
63         ground again after", t0, "seconds."
64     WRITE(UNIT=6, FMT="(A36, X, I2,X, A14)") "The displacement of the
65         particle is", 0, "m from origin."
66     WRITE(UNIT=6, FMT="(A26, X, F5.2,X, A22)") "The distance it
67         covers is", 2*simp, "m in its entire flight."
68
69
70 END PROGRAM problem3_2
71
72
73 FUNCTION f(t) RESULT(s)
74     IMPLICIT NONE
75     REAL :: t, s, v0=10, g=10
76     s = v0*t - g/2*t**2
77 END FUNCTION f
78
79
80 FUNCTION v(t) RESULT(vt)
81     IMPLICIT NONE
82     REAL :: t, vt, v0=10, g=10
83     vt = v0 - g*t
84 END FUNCTION v=====

```

Listing 4: Fortran code for Root Finding and Integration

Results

Based on the computation:

- The time taken to strike back is: $t = 2.00$ s

- The total displacement calculated is: $S \approx \mathbf{9.99994}$ m

Problem 5: Differential Equation Solver

Problem Statement

Consider the differential equation:

$$\frac{d^2x}{dt^2} + \omega_0^2 x = 0$$

Determine the solution using an initial value problem solver. **Conditions:** $x(0) = 0$, $x'(0) = 1$. Plot $x(t)$ vs t .

Fortran Code

```
1  ! =====
2  MODULE ODE_DRIVER
3      CONTAINS
4
5
6  SUBROUTINE corrector(x, y, z, dx)
7      IMPLICIT NONE
8      REAL, EXTERNAL :: f
9      REAL, DIMENSION(0:) :: x, y, z
10     REAL :: dx, k1, k2, k3, p1, p2, p3
11     INTEGER :: i, N
12     N = size(x) - 1
13
14     DO i=1, N - 1
15         x(i) = x(i-1) + dx
16         y(i) = y(i-1) + dx*z(i-1)
17         z(i) = z(i-1) + dx*f(y(i-1), z(i-1))
18
19         k1 = dx*z(i-1)
20         k2 = dx*z(i)
21         p1 = dx*f(y(i-1), z(i-1))
22
23         y(i+1) = y(i-1) + 2*k2
24         p2 = dx*f(y(i), z(i))
25         z(i+1) = z(i-1) + 2*p2
26
27         k3 = dx*z(i+1)
28         p3 = dx*f(y(i+1), z(i+1))
29         y(i+1) = y(i-1) + (k1 + 4*k2 + k3)/3
30         z(i+1) = z(i-1) + (p1 + 4*p2 + p3)/3
31
32     END DO
33
34 END SUBROUTINE corrector
35
36
37 END MODULE ODE_DRIVER
```

```

38
39 PROGRAM problem3_1a
40     USE ODE_DRIVER
41     IMPLICIT NONE
42     INTEGER, PARAMETER :: N=100000
43     REAL :: x(0:N), y_c(0:N), z_c(0:N), dx, a=0, c=1.8
44     INTEGER :: k, i
45
46     ! let  $dy/dx = z$ , and  $a$  &  $c$  the range of the solution
47     ! we also define external function " f " to be used as  $dz/dx$ 
48     ! = f
49
50     dx = (c-a)/(0.1*N)
51     x(0) = 0
52
53     y_c(0) = 0.0
54     z_c(0) = 1
55
56     CALL corrector(x, y_c, z_c, dx)
57
58     OPEN(UNIT=10, FILE="ass_problem5.csv", POSITION="APPEND",
59          ACCESS="SEQUENTIAL", STATUS="REPLACE")
60     DO k=0, N-2
61         WRITE(UNIT=10, FMT="(7f9.5)") x(k), y_c(k), z_c(k)
62     END DO
63     CLOSE(UNIT=10)
64
65 END PROGRAM problem3_1a
66
67 FUNCTION f(y,z) RESULT(s)
68     IMPLICIT NONE
69     REAL :: z, y, s, b=0.0, w0=1, r
70
71
72     s = -(b*z + w0**2*y)
73 END FUNCTION f
74
75 =====

```

Listing 5: Fortran code for ODE Solver

Graph

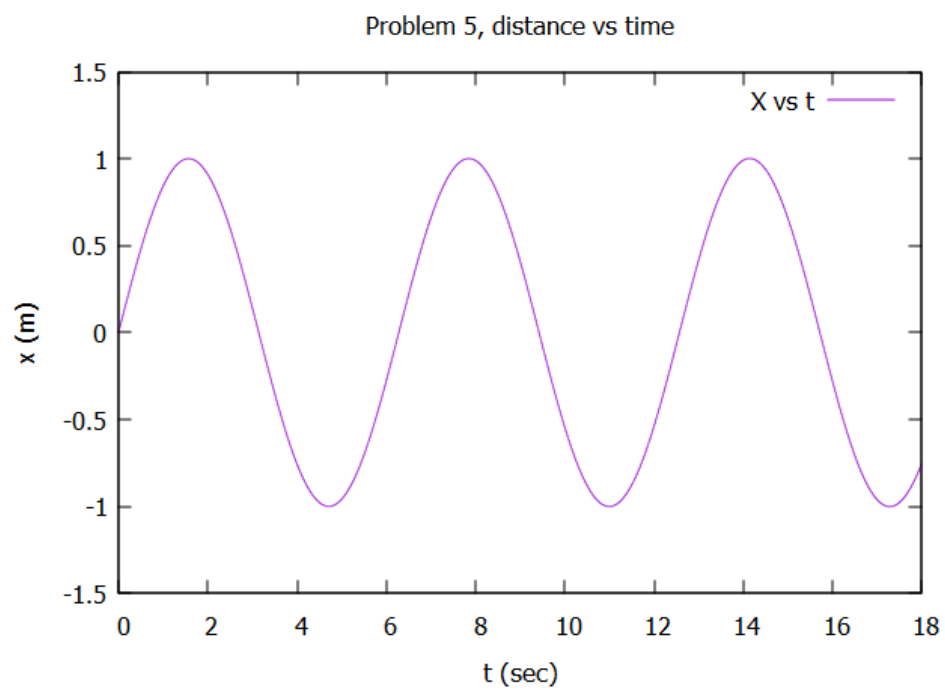


Figure 2: Solution of the harmonic oscillator equation.

Problem 6: O_2 Molecule Energy Analysis

Problem Statement

If problem #5 represents an oscillation of a bond in an O_2 molecule, determine how the total energy E vs t changes by applying a Lennard-Jones (LJ) potential. Use **Molecular Dynamics** (Euler or Verlet) and a **Monte Carlo** scheme.

Preliminary Mathematical Analysis

To determine the equilibrium bond distance, we start with the Lennard-Jones potential given in the problem:

$$V(d) = 4\epsilon \left[\left(\frac{\sigma}{d} \right)^{12} - \left(\frac{\sigma}{d} \right)^6 \right] \quad (1)$$

The force between the atoms is the negative gradient of the potential with respect to the distance d :

$$\begin{aligned} F(d) &= -\frac{dV}{dd} \\ &= -4\epsilon \frac{d}{dd} (\sigma^{12} d^{-12} - \sigma^6 d^{-6}) \\ &= -4\epsilon (-12\sigma^{12} d^{-13} - (-6)\sigma^6 d^{-7}) \\ &= \frac{24\epsilon}{d} \left[2 \left(\frac{\sigma}{d} \right)^{12} - \left(\frac{\sigma}{d} \right)^6 \right] \end{aligned}$$

At equilibrium, the net force is zero ($F(d_{eq}) = 0$). Ignoring the trivial solution at $d \rightarrow \infty$:

$$2 \left(\frac{\sigma}{d_{eq}} \right)^{12} - \left(\frac{\sigma}{d_{eq}} \right)^6 = 0 \quad (2)$$

Let $x = (\sigma/d_{eq})^6$. The equation becomes $2x^2 - x = 0$, which implies $x = 1/2$. Thus:

$$\left(\frac{\sigma}{d_{eq}} \right)^6 = \frac{1}{2} \implies d_{eq} = \sigma \cdot 2^{1/6} \quad (3)$$

Substituting the given parameters $\sigma = 1.1$ and $\epsilon = 1.4$:

$$d_{eq} \approx 1.1 \times 1.12246 \approx 1.2347 \quad (4)$$

This theoretical value $d_{eq} \approx 1.235$ will be used to verify the molecular dynamics simulation results.

Small Oscillations (Harmonic Approximation)

For small displacements $x = d - d_{eq}$, the potential can be approximated as a simple harmonic oscillator:

$$V(d) \approx \frac{1}{2} k (d - d_{eq})^2 \quad (5)$$

where the spring constant k is the second derivative of the potential evaluated at equilibrium:

$$k = \left. \frac{d^2 V}{dd^2} \right|_{d_{eq}} = \frac{4\epsilon}{\sigma^2} \left(\frac{156}{d_{eq}^{12}} - \frac{42}{d_{eq}^6} \right) = \frac{57.15\epsilon}{\sigma^2} = 72.7$$

(6)

This confirms that for small amplitudes, the motion should exhibit simple harmonic behavior, satisfying $E = T + U = \text{constant}$.

Hence from 5 and , we can approximate the force as follows and solve for d:

$$F = -k(d - d_{eq}) = m\ddot{d} \quad (7)$$

Fortran Code

```
1  ! =====
2  MODULE ODE_DRIVER
3      CONTAINS
4
5  SUBROUTINE corrector(t, x, v, dt)
6      IMPLICIT NONE
7      REAL(KIND=16), EXTERNAL :: f
8      REAL(KIND=16), DIMENSION(0:) :: t, x, v
9      REAL(KIND=16) :: dt, k1, k2, k3, p1, p2, p3
10     INTEGER :: i, N
11     N = size(t) - 1
12
13     DO i=1, N - 1
14         t(i) = t(i-1) + dt
15         x(i) = x(i-1) + dt*v(i-1)
16         v(i) = v(i-1) + dt*f(x(i-1))
17
18         k1 = dt*v(i-1)
19         k2 = dt*v(i)
20         p1 = dt*f(x(i-1))
21
22         x(i+1) = x(i-1) + 2*k2
23         p2 = dt*f(x(i))
24         v(i+1) = v(i-1) + 2*p2
25
26         k3 = dt*v(i+1)
27         p3 = dt*f(x(i+1))
28         x(i+1) = x(i-1) + (k1 + 4*k2 + k3)/3
29         v(i+1) = v(i-1) + (p1 + 4*p2 + p3)/3
30
31     END DO
32
33 END SUBROUTINE corrector
34
35
36 END MODULE ODE_DRIVER
37
38 PROGRAM exam2
39     USE ODE_DRIVER
```

```

40  IMPLICIT NONE
41  INTEGER, PARAMETER :: N=10000
42  REAL(KIND=16) :: t(0:N), x(0:N), v(0:N), dt, a=0, c=4e-14
43  REAL(KIND=16) :: E_t(0:N), E_k(0:N), E_p(0:N), xeq=1.235e-10,
      k=72.7, m=35e-27
44  INTEGER :: p, i
45
46  ! let dx/dt = v, and v0 & x0 the initial conditions
47  ! we also define external function " f " to be used as dv/dt
      = f
48
49  dt = (c-a)/(0.1*N)
50  t(0) = 0
51
52  x(0) = 2e-11
53  v(0) = 0
54
55  CALL corrector(t, x, v, dt)
56
57  E_t = 0.5*m*v**2 + 0.5*k*(x-xeq)**2
58  E_k = 0.5*m*v**2
59  E_p = 0.5*k*(x-xeq)**2
60  OPEN(UNIT=10, FILE="home_exam2.csv", POSITION="APPEND",
      ACCESS="SEQUENTIAL", STATUS="REPLACE")
61  DO p=0, N-2
62      WRITE(UNIT=10, FMT="(6f32.20)") t(p), x(p), v(p), E_t(p),
          E_k(p), E_p(p)
63  END DO
64  CLOSE(UNIT=10)
65  PRINT*, x(10), dt
66
67  END PROGRAM exam2
68
69  FUNCTION f(x) RESULT(s)
70      IMPLICIT NONE
71      REAL(KIND=16) :: x, xeq=1.235e-10, s, k=72.7, m=35e-27, r
72      CALL RANDOM_NUMBER(r)
73
74      s = -k*(x - xeq)/m + ((2*r - 1)*1e-7)/m
75  END FUNCTION f
76
77  =====

```

Listing 6: Fortran code for MD Simulation (Oxygen Molecule)

Results and Graphs

In delivering graphical results for problem 2, I have used the following necessary initial conditions and constants, as can be shown in the above code.

Symbol	Description	Value
m	Mass of an oxygen molecule	$5.31\text{e-}26$
k	Force constant (calculated)	72.7
x_{eq}	Equilibrium position	$1.21\text{e-}10$
x_0	Initial displacement from equilibrium	$2.0\text{e-}11$
t_0	Initial time	0

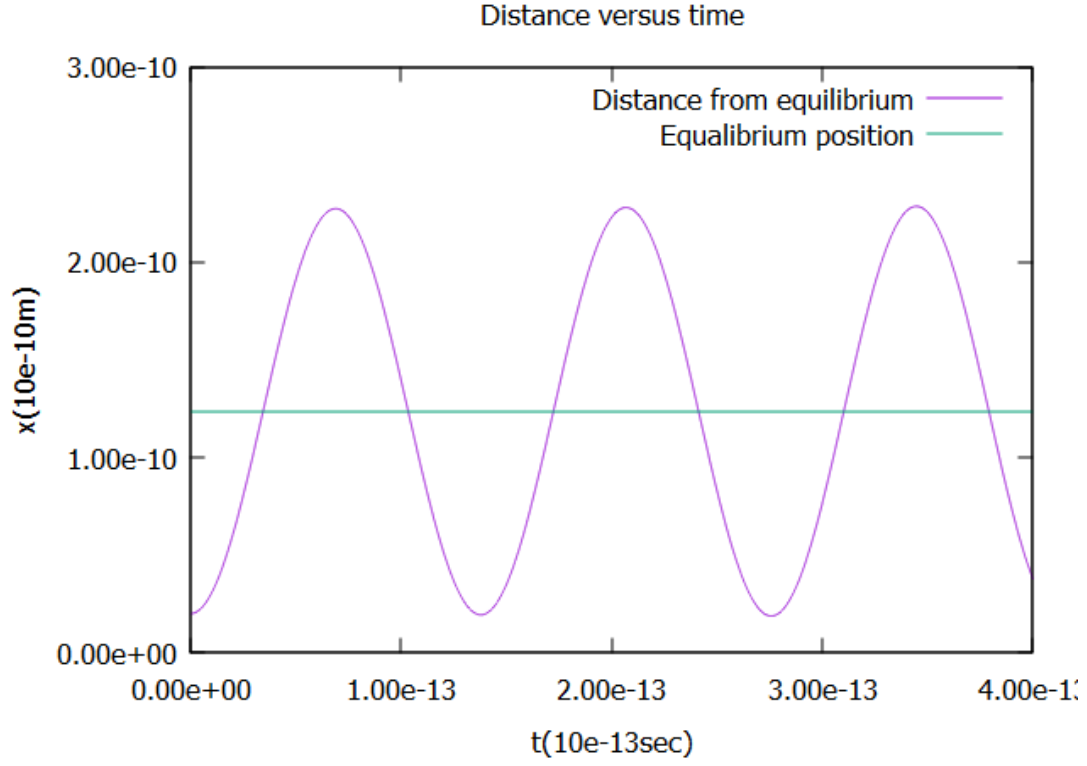


Figure 3: Bond distance d as a function of time.

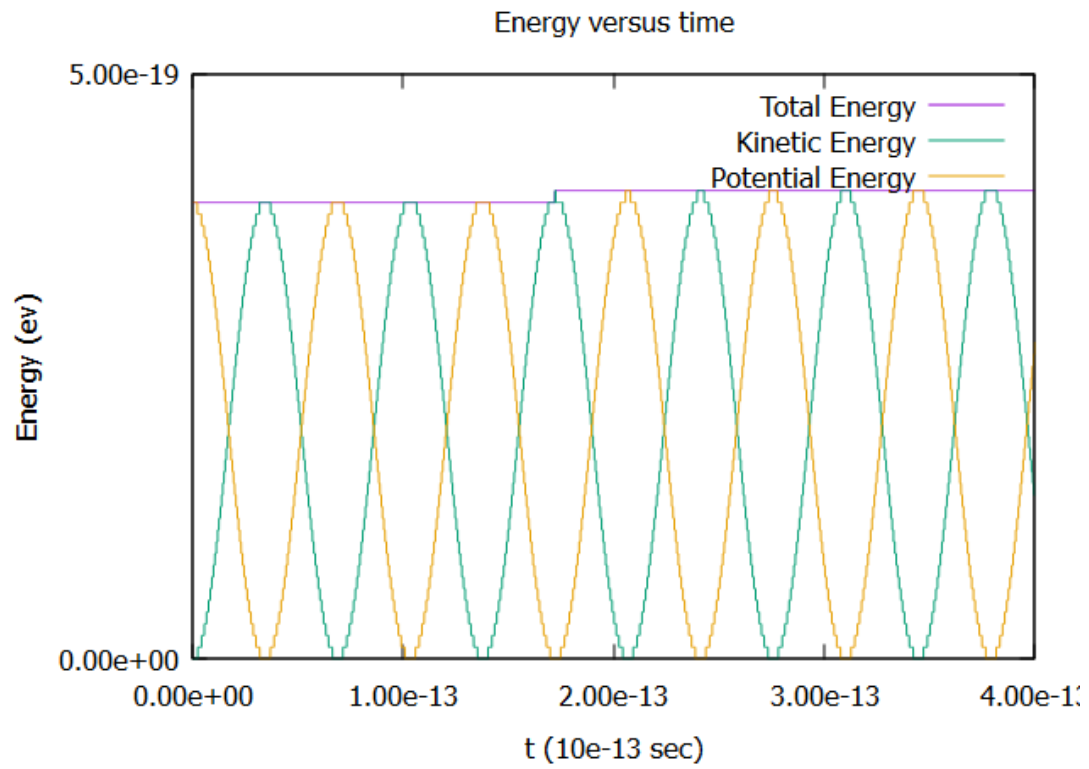


Figure 4: The plot of total energy, potential energy, and kinetic energy as a function of time.